

Capstone Project Brief

Multi-Agent AI Product using LangGraph

Course: Multi Agent Orchestration [AI/ML]

Team Size	Project Type	Presentation	Evaluation Week
3-5 students	Group project	10 min + 5 min Q&A	25-30 June

1. Capstone Objective

Build a working multi-agent AI product using LangGraph or an equivalent orchestration framework. The project should solve a complex, meaningful problem and demonstrate clear agent roles, tool use, state flow, evaluation, debugging, guardrails, and safe product behavior.

2. Team Formation and Submission Requirements

Group Size

Students must form groups of 3 to 5 students. Each group will submit one project and present it during the exam-week evaluation.

Every student must clearly present their individual contribution during the evaluation.

Additionally, every student must individually submit the Google Form with:

- The Project GitHub Repository link
- An Individual Contribution Document outlining their specific contributions to the project

3. Problem Statement Selection

Groups are free to select any meaningful, interesting, and complex problem that requires a multi-agent architecture. **The selected problem should not be a simple chatbot or one-prompt application.** It should involve multiple roles, structured handoffs, tool usage, decision-making, evaluation, and review.

For inspiration, students may refer to YC-funded AI Assistant companies:

<https://www.ycombinator.com/companies/industry/ai-assistant>

4. Minimum Technical Requirements

Requirement	Expectation
Multi-agent orchestration	At least 3 meaningful agents with distinct responsibilities
LangGraph or equivalent	LangGraph preferred; CrewAI, AutoGen, LlamaIndex Workflows, or similar allowed with justification
State management	Maintain shared graph/system state across steps
Tool use	At least 2 tools, APIs, or integrations
Structured outputs	Use JSON/Pydantic/schema-based outputs for important agent handoffs
Routing or branching	Include at least one conditional decision in the system
RAG or knowledge grounding	Use retrieval where it improves grounding, or justify why RAG is not needed
Evaluation	Include at least 5 test cases or evaluation scenarios
Debugging / observability	Use LangSmith traces or clear logs/intermediate outputs
Guardrails	Include validation, policy checks, refusal handling, or safety boundaries
Human-in-the-loop	Required for high-impact actions such as sending messages, updating records, approving recommendations, or taking external actions
Demo-ready output	The system should run end-to-end with sample inputs

5. Presentation and Evaluation

Evaluation will happen during exam week: 25th June - 30th June. Slots will be shared with each group.

Each group will get 10 minutes for presentation and 5 minutes for Q&A. Every student must participate and explain their individual contribution.

Presentation Structure

Time	What to cover
0-1 min	Problem statement, target user, and why the problem matters
1-2 min	Why the problem requires a multi-agent system
2-4 min	Architecture: agents, tools, state, routing, and handoffs
4-7 min	Working demo of the system
7-8.5 min	Evaluation cases, guardrails, debugging, and limitations
8.5-10 min	Individual contributions of each group member
10-15 min	Q&A with evaluators

6. Evaluation Rubric

Suggested total: 100 marks

Criterion	Weight	What will be judged
Problem selection and product clarity	10%	Is the problem meaningful, clearly explained, and complex enough to justify multi-agent design?
Multi-agent architecture	20%	Are the agents meaningfully specialized with clear roles, responsibilities, and handoffs?
LangGraph implementation	15%	Are state, nodes, edges, routing, branching, and graph flow implemented correctly?
Tool use and integrations	10%	Are tools, APIs, RAG, or integrations used meaningfully rather than artificially?
State, memory, and context design	10%	Is information passed, stored, retrieved, and reused cleanly across the system?
Evaluation and debugging	10%	Are there test cases, traces/logs, failure analysis, and meaningful improvements?
Guardrails and human-in-the-loop	10%	Are risky actions controlled using policy checks, validation, refusal, or approval?
Demo quality and usability	10%	Does the project run end-to-end and produce a useful, clear, demo-ready output?
Individual contribution clarity	15%	Can each student clearly explain their own contribution, design decisions, implementation work, and how their part connects to the full system?

7. Important Rules

- The project must not be a simple chatbot.
- The project must include at least 3 meaningful agents with distinct responsibilities.
- Tools should be assigned intentionally and should support the product workflow.
- RAG should be used only where grounding is useful.
- High-impact external actions must include human approval.
- The final system should be demo-ready and runnable with sample inputs.
- Every student must be ready to explain their contribution during the presentation/viva.
- YC references are only for inspiration; students must build their own original implementation.